

Bitwise Operators in Java

 [geeksforgeeks.org/bitwise-operators-in-java](https://www.geeksforgeeks.org/bitwise-operators-in-java)

August 8, 2018

Bitwise Operators

Bitwise operators are used to performing the manipulation of individual bits of a number. They can be used with any integral type (char, short, int, etc.). They are used when performing update and query operations of the Binary indexed trees.

Now let's look at each one of the bitwise operators in Java:

1. Bitwise OR (|)

This operator is a binary operator, denoted by '|'. It returns bit by bit OR of input values, i.e., if either of the bits is 1, it gives 1, else it shows 0.

Example:

```
a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)
```

Bitwise OR Operation of 5 and 7

```
  0101
| 0111
```

```
0111 = 7 (In decimal)
```

2. Bitwise AND (&)

This operator is a binary operator, denoted by '&'. It returns bit by bit AND of input values, i.e., if both bits are 1, it gives 1, else it shows 0.

Example:

```
a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)
```

Bitwise AND Operation of 5 and 7

```
  0101
& 0111
```

```
0101 = 5 (In decimal)
```

3. Bitwise XOR (

This operator is a binary operator, denoted by '^.' It returns bit by bit XOR of input values, i.e., if corresponding bits are different, it gives 1, else it shows 0.

Example:

a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise XOR Operation of 5 and 7

0101
^ 0111

0010 = 2 (In decimal)

4. Bitwise Complement (~)

This operator is a unary operator, denoted by '~.' It returns the one's complement representation of the input value, i.e., with all bits inverted, which means it makes every 0 to 1, and every 1 to 0.

Example:

a = 5 = 0101 (In Binary)

Bitwise Complement Operation of 5

~ 0101

1010 = 10 (In decimal)

Note: Compiler will give 2's complement of that number, i.e., 2's complement of 10 will be -6.

```
// Java program to illustrate
// bitwise operators

public class operators {

    public static void main(String[] args)

    {

        // Initial values

        int a = 5;

        int b = 7;
```

```

// bitwise and
// 0101 & 0111=0101 = 5
System.out.println("a&b = " + (a & b));

// bitwise or
// 0101 | 0111=0111 = 7
System.out.println("a|b = " + (a | b));

// bitwise xor
// 0101 ^ 0111=0010 = 2
System.out.println("a^b = " + (a ^ b));

// bitwise not
// ~00000000 00000000 00000000 00000101=11111111 11111111 11111111 11111010
// will give 2's complement (32 bit) of 5 = -6
System.out.println("~a = " + ~a);

// can also be combined with
// assignment operator to provide shorthand
// assignment
// a=a&b
a &= b;

System.out.println("a= " + a);
}
}

```

Output

```

a&b = 5
a|b = 7
a^b = 2
~a = -6
a= 5

```

Auxiliary space: $O(1)$

Time complexity: $O(1)$

```

// Demonstrating the bitwise logical operators

class GFG {

    public static void main (String[] args) {

        String binary[]={
            "0000", "0001", "0010", "0011", "0100", "0101",
            "0110", "0111", "1000", "1001", "1010",
            "1011", "1100", "1101", "1110", "1111"
        };

        // initializing the values of a and b
        int a=3; // 0+2+1 or 0011 in binary
        int b=6; // 4+2+0 or 0110 in binary

        // bitwise or
        int c= a | b;

        // bitwise and
        int d= a & b;

        // bitwise xor
        int e= a ^ b;

        // bitwise not
        int f= (~a & b)|(a & ~b);
        int g= ~a & 0x0f;
    }
}

```

```

System.out.println(" a= "+binary[a]);
System.out.println(" b= "+binary[b]);
System.out.println(" a|b= "+binary);
System.out.println(" a&b= "+binary[d]);
System.out.println(" a^b= "+binary[e]);
System.out.println("~a & b|a&~b= "+binary[f]);
System.out.println("~a= "+binary[g]);
}
}

```

Output

```

a= 0011
b= 0110
a|b= 0111
a&b= 0010
a^b= 0101
~a & b|a&~b= 0101
~a= 1100

```

Bit-Shift Operators (Shift Operators)

Shift operators are used to shift the bits of a number left or right, thereby multiplying or dividing the number by two, respectively. They can be used when we have to multiply or divide a number by two.

Syntax:

```
number shift_op number_of_places_to_shift;
```

Types of Shift Operators:

Shift Operators are further divided into 4 types. These are:

1. Signed Right shift operator (>>)
2. Unsigned Right shift operator (>>>)
3. Left shift operator(<<)
4. Unsigned Left shift operator (<<<)

program to implement all Bitwise operators in java for user input

```
import java.util.Scanner;

public class BitwiseOperators {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.print("Enter first number: ");

        int num1 = input.nextInt();

        System.out.print("Enter second number: ");

        int num2 = input.nextInt();

        System.out.println("Bitwise AND: " + (num1 & num2));

        System.out.println("Bitwise OR: " + (num1 | num2));

        System.out.println("Bitwise XOR: " + (num1 ^ num2));

        System.out.println("Bitwise NOT: " + (~num1));

        System.out.println("Bitwise Left Shift: " + (num1 << 2));

        System.out.println("Bitwise Right Shift: " + (num1 >> 2));

        System.out.println("Bitwise Unsigned Right Shift: " + (num1 >>> 2));

        input.close();

    }

}
```

Input

Enter first number: 4
Enter second number: 8

Output

Bitwise AND: 0
Bitwise OR: 12
Bitwise XOR: 12
Bitwise NOT: -5
Bitwise Left Shift: 16
Bitwise Right Shift: 1
Bitwise Unsigned Right Shift: 1

Explanation

This program prompts the user to enter two numbers, num1 and num2. It then performs the following bitwise operations using the &, |, ^, ~, <<, >>, and >>> operators:

Advantages

The advantages of using Bitwise Operators in Java are:

1. **Speed:** Bitwise operations are much faster than arithmetic operations as they operate directly on binary representations of numbers.
2. **Space Optimization:** Bitwise operations can be used to store multiple values in a single variable, which can be useful when working with limited memory.
3. **Bit Manipulation:** Bitwise operators allow for precise control over individual bits of a number, which can be useful in various applications such as cryptography, error detection, and compression.
4. **Code Simplification:** Bitwise operations can simplify the code by reducing the number of conditional statements and loops required to perform certain tasks.
5. **Improved Readability:** Bitwise operations can make the code more readable by encapsulating complex logic into a single operation, making the code easier to understand and maintain.

In summary, Bitwise Operators are an important tool for optimizing performance, improving code readability, and reducing code complexity in Java applications.

Last Updated : 17 Apr, 2023